

05 | 核心交互模型：所有Agent的通用语言——上下文注入与Shell执行

Tony Bai · AI原生开发工作流实战



你好，我是 Tony Bai。

从这一讲开始，我们将正式进入 Claude Code 的实战世界。在前面的概念篇（01~03 讲），我们建立了宏大的认知框架；在上一讲，我们亲手搭建好了开发环境。现在，是时候拿起“武器”，学习如何驾驭我们这位强大的 AI 原生伙伴了。

要真正驾驭我们这位强大的 AI 原生伙伴，关键在于升级我们的交互模式。与高级 AI Agent 最有效的协作，早已超越了简单的“一问一答”；它是一场**结构化的、双向的对话**，在这场对话中，我们不仅要清晰地表达意图，更要高效地提供上下文，并授权其采取行动。

这正是**结构化交互**的威力所在。一个设计精良的 Agent，其目标是降低你的认知负荷，而不是要求你成为“提示词大师”。它会提供一套简洁而强大的“交互模型”，让你能够轻松地表达复杂的工程意图。

而 Claude Code（以及大多数优秀的 Coding Agent）的核心交互模型，就建立在两个极其简单却异常强大的符号之上：`@` 和 `!`。

`@` 代表**上下文 (Context)**，是 AI 用来“感知”世界的眼睛和耳朵。

`!` 代表**行动 (Action)**，是 AI 用来“改造”世界的双手。

这一讲，我们的目标就是彻底精通 AI 的这两只“手”。你将发现，这两个简单的符号，不仅是操作指令，更是一种深刻的人机协作哲学。掌握它们，你就掌握了与所有高级 Coding Agent 对话的“通用语言”。

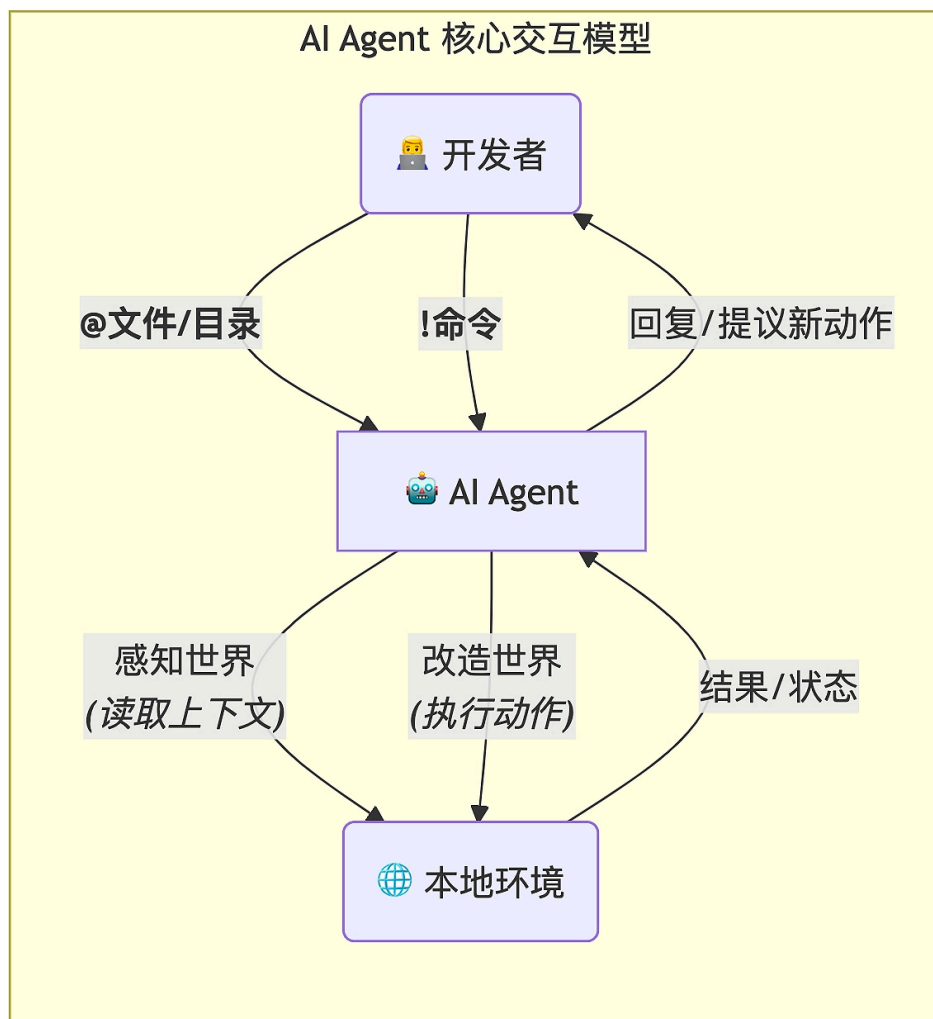
AI 的两只手：一种新的人机交互哲学

在传统的软件开发中，我们与计算机的交互是单向的、命令式的。我们编写代码，编译器执行它。而在 AI 原生开发的范式中，这种交互变得更像是**双向的、协作式的对话**。

想象一下你正在与一位人类的资深架构师同事结对编程。你们的协作通常是这样的：

1. 你（指着屏幕）：“你看一下这段代码（`@`），我觉得它的耦合度太高了。”（提供上下文）
2. 同事（阅读代码后）：“确实。我建议把数据访问逻辑抽离出来。你先**运行一下测试**（`!`），确保我们现在的状态是稳定的。”（提议行动）
3. 你运行了测试。
4. 同事：“好的，测试通过。现在，我来帮你重构。我将要**修改这几个文件**（`!`），把数据库调用移到一个新的 `repository` 包里。”（提议行动）

`@` 和 `!`，正是将这种高效的人类协作模式，完美地复刻到了人机交互之中。



@ 指令：它将“提供上下文”这个动作，从一个模糊的、需要靠复制粘贴完成的任务，变成了一个**精确的、可追溯的、结构化的指令**。当你输入 `@main.go` 时，你和 AI 之间就建立了一个明确的契约：“我们接下来的对话，是围绕`main.go`这个文件的当前状态展开的。”

! 指令：它将“执行动作”这个环节，从一个需要你切换窗口、手动操作的流程，无缝地**融入到了对话流之中**。它打通了 AI 的“大脑”（语言模型）和你的本地环境之间的“隔膜”，让 AI 的思考可以直接转化为真实世界的影响。

接下来，让我们深入每一个指令的细节，看看它们究竟有多强大。

精通 @ 指令：为 AI 装上“全景摄像头”

@ 指令的核心使命，是解决我们在开篇词中提到的“上下文摩擦力”问题。它让 AI 不再是一个“健忘的外部顾问”，而是成为了一个能够主动观察你整个项目的“全局架构师”。

下面我们就来看看 @ 指令的几种常见用法。

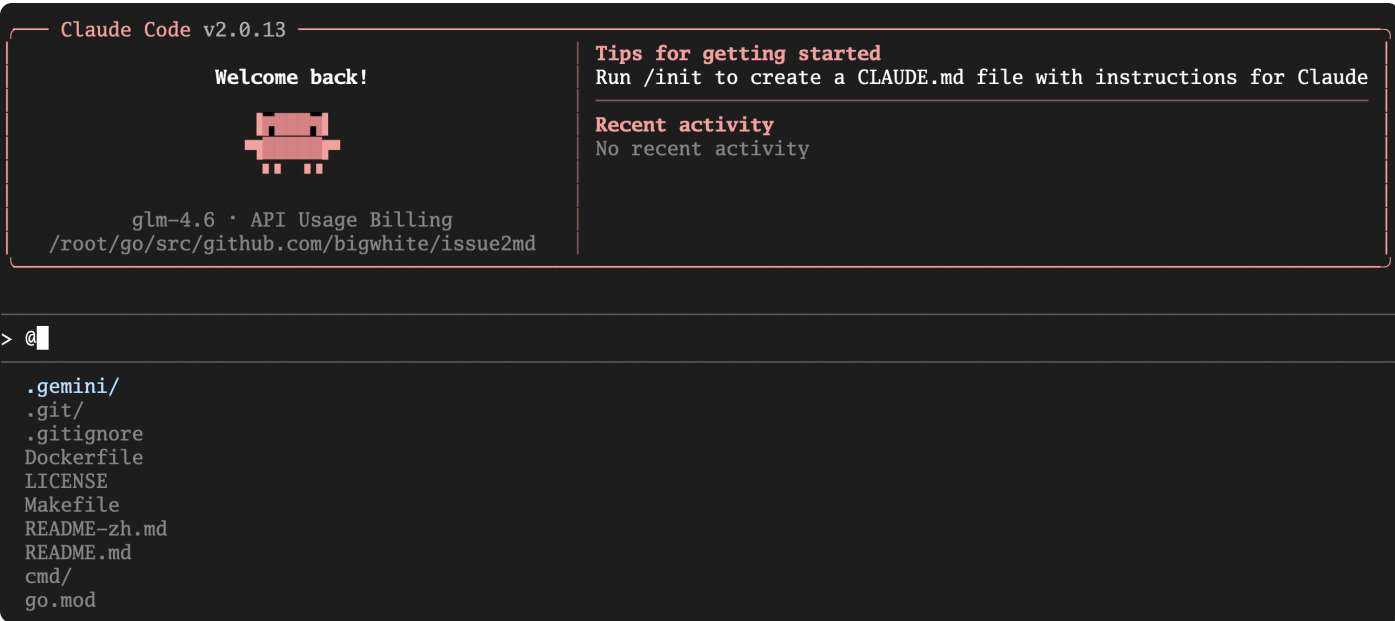
引用单个文件：最精准的上下文

这是 @ 指令最基础也是最高频的用法。你可以用它来引用项目中的任何一个文件，无论是源代码、配置文件、文档，还是测试数据。

 复制代码

```
1 语法: @/path/to/your/file
```

Claude Code 拥有强大的路径自动补全功能。你只需输入 @，就会自动列出当前目录下的文件和目录列表（如下图所示）；输入和文件名的前几个字母，然后按下 Tab 键，它也会像 IDE 一样为你补全路径。



接下来，我们将结合开发者日常工作，看两个 @ 指令与单个文件结合的实战场景。

实战场景 1：代码解释

假设你正在接手一个陌生的 Go 项目，看到了一个复杂的 Dockerfile 文件，你想快速理解它的作用。你可以直接将 Dockerfile 通过 @ 指令喂给 Claude Code：


```
> @Dockerfile
└─ Read Dockerfile (37 lines)

● I can see the Dockerfile for your issue2mdweb project. This is a multi-stage Docker build configuration that creates a containerized version of your Go web application.

The Dockerfile has these main stages:

1. Build Stage (lines 1-17):
  - Uses Go 1.21 image
  - Downloads dependencies with go mod tidy
  - Builds the issue2mdweb binary
2. Production Stage (lines 19-36):
  - Uses lightweight Alpine Linux
  - Copies the compiled binary and web assets
  - Exposes port 8080
  - Sets the entrypoint to run the application

The configuration looks clean and follows Docker best practices with multi-stage builds to minimize the final image size.

> █
? for shortcuts
```

我们看到，Claude Code 在加载 Dockerfile（我使用的是 bigwhite/issue2md 项目下的 Dockerfile）时，会输出对该上下文数据的概要理解。当然如果你要进一步针对该 Dockerfile 提问，可以直接输入你的问题：

 复制代码

```
1 > 帮我逐行解释一下这个Dockerfile的作用，特别是多阶段构建的逻辑。
2
3 ● 我来为您详细解释这个Dockerfile的每一行以及多阶段构建的逻辑：
4
5 第一阶段：构建阶段（Build Stage）
6
7 # 使用 Go 语言官方镜像作为构建阶段
8 FROM golang:1.21 AS builder
9 - 使用官方Go 1.21镜像作为基础镜像
10 - AS builder 为这个阶段命名为"builder"，便于后续引用
11
12 # 设置工作目录
13 WORKDIR /app
14 - 在容器内创建/app目录并设置为工作目录
15 - 后续所有命令都会在此目录下执行
16
17 # 复制 go.mod文件
18 COPY go.mod ./
19 - 只复制go.mod文件到工作目录
20 - 这是Docker层缓存的优化策略
21
22 # 下载依赖
23 RUN go mod tidy
24 - 下载项目所需的Go模块依赖
25 - 只复制go.mod就下载依赖可以利用Docker缓存，如果go.mod没变就不会重新下载
```

```
26 # 复制源代码
27 COPY . .
28 - 复制项目所有源代码到工作目录
29 - 在依赖下载之后执行，确保代码变更时才重新构建
30
31 # 构建二进制文件
32 RUN CGO_ENABLED=0 GOOS=linux go build -o issue2mdweb .
33 - CGO_ENABLED=0: 禁用CGO，生成静态链接的二进制文件
34 - GOOS=linux: 指定目标操作系统为Linux
35 - -o issue2mdweb: 指定输出文件名
36 - 编译当前目录(.)的Go代码
37
38 第二阶段：生产阶段 (Production Stage)
39
40 # 使用轻量级的镜像作为生产阶段
41 FROM alpine:latest
42 - 使用轻量级的Alpine Linux作为生产环境基础镜像
43 - Alpine镜像体积很小，适合生产环境
44
45 # 设置工作目录
46 WORKDIR /root/
47 - 设置生产容器的工作目录为/root/
48
49 # 从构建阶段复制二进制文件
50 COPY --from=builder /app/issue2mdweb .
51 - --from=builder: 从名为"builder"的构建阶段复制文件
52 - 将编译好的二进制文件复制到当前工作目录
53
54 # 复制 web 目录下的 templates 和 static 子目录
55 COPY --from=builder /app/web/templates /root/web/templates
56 COPY --from=builder /app/web/static /root/web/static
57 - 分别复制Web模板文件和静态资源文件
58 - 保持原有的目录结构
59
60 # 暴露应用端口
61 EXPOSE 8080
62 - 声明容器监听8080端口
63 - 这是文档声明，实际端口映射需要在运行时指定
64
65 # 指定容器启动时的命令
66 ENTRYPOINT ["/issue2mdweb"]
67 - 设置容器启动时执行的命令
68 - 运行编译好的Go应用程序
69
70 多阶段构建的优势
71
72 1. 减小镜像体积：最终镜像只包含运行时必需的文件，不包含Go编译器和源代码
73 2. 提高安全性：生产镜像不包含构建工具，减少攻击面
74
```

```
75 3. 提高构建效率：利用Docker层缓存，只有依赖变更时才重新下载
76 4. 清晰的分离：构建环境和运行环境分离，便于维护
77
78 这种模式是现代Go应用容器化的最佳实践
```

注意：你可以将 `@Dockerfile` 和你的问题放在同一行，也可以像我这样分两次输入，效果是一样的。

我们看到 Claude Code 会读取 `Dockerfile` 的完整内容，并给出一份详尽的、逐行分析的
解释。你不再需要将几十行配置手动复制到 Claude Code 中了。

实战场景 2：基于代码生成测试

你刚刚写完一个 Go 函数，现在需要为它编写单元测试。

`utils/math.go`

 复制代码

```
1 package utils
2
3 func Add(a, b int) int {
4     return a + b
5 }
```

你可以这样请求 AI：

 复制代码

```
1 > @utils/math.go
2 > 为这个文件里的 Add 函数编写一份完整的单元测试，使用Go语言的testing包，并覆盖正常、负数和零这
```

AI 会读取 `math.go`，理解 `Add` 函数的签名和功能，然后为你输出一份高质量的
`utils/math_test.go` 文件内容：

```
> @utils/utils.go 为这个文件里的 Add 函数编写一份完整的单元测试，使用Go语言的testing包，并覆盖正常、负数和零这几种情况。  
└─ Read utils/utils.go (6 lines)
```

```
● Write(utils/utils_test.go)
```

Create file

```
utils/utils_test.go  
  
package utils  
  
import "testing"  
  
func TestAdd(t *testing.T) {  
    tests := []struct {  
        name string  
        a     int  
        b     int  
        want int  
    }{  
        {  
            name: "正常情况_两个正数相加",  
            a:    3,  
            b:    5,  
            want: 8,  
        },  
        {  
            name: "正常情况_正数加零",  
            a:    10,  
            b:    0,  
            want: 10,  
        },  
        {  
            name: "正常情况_零加正数",  
            a:    0,  
            b:    7,  
        },  
    }  
    for _, test := range tests {  
        t.Run(test.name, func(t *testing.T) {  
            result := Add(test.a, test.b)  
            if result != test.want {  
                t.Errorf("Add(%d, %d) = %d, want %d", test.a, test.b, result, test.want)  
            }  
        })  
    }  
}
```

在生成的内容末尾，Claude Code 还给出了三个授权选项：

Do you want to create **utils_test.go**?

- > 1. Yes
- 2. Yes, allow all edits during this session (shift+tab)
- 3. No, and tell Claude what to do differently (esc)

选项 1 是授权 Claude Code 在本地创建该 `utils/math_test.go` 文件，选项 2 是授权 Claude Code 在本次会话下可以进行任何编辑操作。考虑安全性，这次我们选择了选项 1，授权 Claude Code 在本地创建 `utils/math_test.go` 文件。之后 Claude Code 将创建该文件并输出如下内容：

- 我已经为 `Add` 函数创建了完整的单元测试文件 `utils_test.go`。这个测试覆盖了以下情况：

测试覆盖范围

正常情况：

- 两个正数相加 ($3 + 5 = 8$)
- 正数加零 ($10 + 0 = 10$)
- 零加正数 ($0 + 7 = 7$)

负数情况：

- 两个负数相加 ($-3 + -5 = -8$)
- 负数加零 ($-10 + 0 = -10$)
- 零加负数 ($0 + -7 = -7$)

混合情况：

- 正数加负数，正数较大 ($10 + -3 = 7$)
- 正数加负数，负数较大 ($3 + -10 = -7$)

边界情况：

- 零加零 ($0 + 0 = 0$)
- 大数相加 ($1000000 + 2000000 = 3000000$)

测试特点

1. 表格驱动测试：使用结构体切片定义测试用例，便于添加新的测试场景
2. 子测试：使用 `t.Run()` 为每个测试用例创建独立的子测试，便于定位问题
3. 清晰的错误信息：当测试失败时，会显示输入值、期望值和实际值
4. 完整的测试命名：每个测试用例都有描述性的名称，便于理解测试目的

您可以通过以下命令运行测试：

```
cd utils
go test -v
```

或者运行覆盖率测试：

```
go test -cover
```

我们看到 Claude Code 还贴心地对测试的覆盖范围做了说明，并给出了后续的建议操作！你可以在本地跑测试，也可以在 Claude Code 中使用 `!` 指令，进入 `utils` 并运行 `go test -v`，以查看测试是否运行通过（如下图）。关于 `!` 指令，我们后面会详细说明。


```
! cd utils
  | (No content)
```

```
> go test -v
```

- 很好！您已经成功运行了测试。让我查看一下测试结果，并检查是否一切正常：

- **Bash(go test -v)**
 - | === RUN TestAdd
 - | === RUN TestAdd/正常情况_两个正数相加
 - | === RUN TestAdd/正常情况_正数加零
 - | ... +21 lines (ctrl+o to expand)

- 完美！所有测试都通过了。让我们也运行一下覆盖率测试来查看测试覆盖率：

引用整个目录：开启“上帝视角”

当你的问题涉及到多个文件或需要宏观视角时，引用单个文件就显得力不从心了。此时，@ 指令的真正威力才显现出来——你可以引用整个目录。

 复制代码

```
1 语法： @/path/to/your/directory/
```

当 AI“看到”一个目录时，它并不会把所有文件的内容都无脑地塞进上下文。Claude Code 的 @ 指令在处理目录时，表现得极其智能。

默认情况下，@ 指令是 **Git 感知**的。这意味着它会：

1. 自动查找并读取你项目根目录下的 `.gitignore` 文件。
2. 在遍历你指定的目录时，**自动忽略**所有匹配 `.gitignore` 规则的文件和目录。

你完全不用担心它会把 `node_modules/`、`vendor/`、编译产物或 `.git/` 目录本身这些“上下文噪音”喂给模型。它提供给 AI 的，永远是干净、纯粹的源代码和重要资产。

实战场景 3：项目概览与技术栈分析

你刚刚克隆了一个新的开源 Go 项目，想快速上手。我们可以将当前目录作为上下文喂给 Claude Code：

```
> @./
```

- **Bash(ls -la)**
 - └ total 20964
 - drwxr-xr-x 9 root root 4096 Oct 13 06:48 .
 - drwxr-xr-x 10 root root 4096 Aug 21 07:29 ..
 - ... +18 lines (ctrl+o to expand)
- 现在您位于 /root/go/src/github.com/bigwhite/issue2md 目录。这是 issue2md 项目的根目录，包含了完整的 Go 项目结构。

主要文件和目录：

- 编译后的二进制文件：issue2md (8.7MB) 和 issue2mdweb (12.6MB)
- 源代码目录：cmd/, internal/, utils/, web/
- 配置文件：go.mod, Makefile, Dockerfile
- 文档：README.md, README-zh.md, LICENSE
- 其他：.git/, .gemini/, scripts/, 屏幕截图等

如果您需要查看或修改项目的任何部分，请告诉我您的需求！

然后，我们向 Claude Code 表述意图，Claude Code 会结合整个目录上下文给出答案：

 复制代码

```
1 > 这是一个Go项目，请帮我分析它的整体结构、核心功能、主要依赖以及如何运行起来。
2
3 ● Read(README.md)
4   └ Read 56 lines
5
6 ● Read(go.mod)
7   └ Read 4 lines
8
9 ● Read(main.go)
10  └ Read 31 lines
11
12 ● Read(Makefile)
13   └ Read 19 lines
14
15 ● Search(pattern: "cmd/**/*.go")
16   └ Found 1 file (ctrl+o to expand)
17
18 ● Search(pattern: "internal/**/*.go")
19   └ Found 2 files (ctrl+o to expand)
20
21 ● Search(pattern: "web/**/*.go")
22   └ Found 1 file (ctrl+o to expand)
23
24 ● Read(cmd/issue2md/main.go)
25   └ Read 123 lines
```

```
26 • Read(internal/converter/converter.go)
27   | Read 116 lines
28
29 • Read(internal/github/github.go)
30   | Read 406 lines
31
32 • Read(web/handlers/handlers.go)
33   | Read 106 lines
34
35 • Search(pattern: "web/**/*.html")
36   | Found 1 file (ctrl+o to expand)
37
38 • Bash(go list -m all)
39   | github.com/bigwhite/issue2md
40
41 • issue2md 项目分析报告
```

项目整体结构

这是一个GitHub Issue/Discussion/PR 转 Markdown工具，采用标准Go项目结构：

```
47 issue2md/
48 |— cmd/                # 命令行工具入口
49 |   |— issue2md/       # CLI工具主程序
50 |— internal/          # 内部包
51 |   |— converter/     # Markdown转换逻辑
52 |   |— github/        # GitHub API交互
53 |— web/               # Web界面
54 |   |— handlers/      # HTTP处理器
55 |   |— templates/     # HTML模板
56 |   |— static/        # 静态资源
57 |— utils/             # 工具函数
58 |— main.go            # Web服务主入口
59 |— Dockerfile         # 容器化配置
60 |— Makefile           # 构建脚本
```

核心功能

1. 双模式运行

- CLI模式 (cmd/issue2md/main.go:25-122)：命令行工具，支持批量转换
- Web模式 (main.go:12-30)：Web服务，提供图形化界面

2. 支持的GitHub资源类型

- Issues (internal/github/github.go:128-162)
- Pull Requests (internal/github/github.go:164-198)
- Discussions (internal/github/github.go:305-335)

```
75 3. 转换功能
76
77 - 结构化Markdown输出 (internal/converter/converter.go:55-115)
78 - 用户链接 (internal/converter/converter.go:13-18)
79 - 反应统计 (internal/converter/converter.go:30-53)
80 - 评论整理 (internal/converter/converter.go:21-27)
81
82 4. GitHub API集成
83
84 - URL解析 (internal/github/github.go:87-117)
85 - 认证支持 (GITHUB_TOKEN环境变量)
86 - 分页处理 (internal/github/github.go:206-263)
87 - Reaction获取 (internal/github/github.go:295-303)
88
89 📦 主要依赖关系
90
91 无外部依赖
92
93 项目仅使用Go标准库:
94 - net/http - HTTP客户端和服务端
95 - encoding/json - JSON数据处理
96 - html/template - Web模板渲染
97 - flag - 命令行参数解析
98 - os, io, fmt - 系统操作
99
100 自定义包结构
101
102 github.com/bigwhite/issue2md
103 |— internal/converter      # 转换器包
104 |— internal/github         # GitHub API包
105 |— web/handlers            # Web处理器包
106
107 🏃 如何运行
108
109 1. CLI模式
110
111 # 安装
112 go install github.com/bigwhite/issue2md/cmd/issue2md@latest
113
114 # 使用
115 issue2md [flags] <github-url> [output-file]
116
117 # 示例
118 issue2md -enable-reactions https://github.com/user/repo/issues/123 issue.md
119
120 2. Web模式
121
122 # 编译
123 make web
```

```
124 # 或
125 go build -o issue2mdweb
126
127 # 运行
128 ./issue2mdweb
129 # 服务器运行在 http://0.0.0.0:8080
130
131 # 使用Docker (推荐)
132 docker run -d -p 8080:8080 bigwhite/issue2mdweb
133
134 3. 构建选项
135
136 # 构建所有版本
137 make all
138
139 # 仅构建CLI
140 make cmdline
141
142 # 构建Docker镜像
143 make buildimage
144
145 # 清理构建产物
146 make clean
147
148 🌀 配置选项
149
150 命令行参数
151
152 - --enable-reactions: 包含反应统计
153 - --enable-user-links: 启用用户资料链接
154 - --ip, -port: Web服务绑定地址和端口
155
156 环境变量
157
158 - GITHUB_TOKEN: GitHub API认证令牌 (提高API限制)
159
160 🌟 项目特点
161
162 1. 轻量级: 无外部依赖, 仅使用标准库
163 2. 双模式: 支持CLI和Web两种使用方式
164 3. 完整支持: 覆盖Issue、PR、Discussion三种类型
165 4. 容器化: 提供Docker部署方案
166 5. 功能丰富: 支持反应统计、用户链接等高级功能
167
168 这是一个设计良好的Go项目, 代码结构清晰, 功能完整, 适合作为学习Go项目结构和GitHub API集成的参
169
```


我们看到 Claude Code 会扫描整个项目（已忽略 `.gitignore` 中的内容），阅读 `go.mod` 来分析依赖，阅读 `Makefile` 或 `main.go` 来寻找启动方式，并阅读 `README.md` 来理解项目目标。最终，它会给你一份堪比资深同事为你做的项目介绍。

实战场景 4：大规模代码重构

假设你想对 `issue2md` 项目中的 `internal/github/` 包进行重构，将所有调用 Github RESTful API 的地方换为基于 Github GraphQL API 的调用，我们可以基于 Github 这个目录的上下文提出我们的重构要求：

```
> @internal/github/
└─ Listed directory internal/github/
```

- **Bash**(ls -la internal/github/)
 - └─ total 20
 - drwxr-xr-x 2 root root 4096 Oct 13 07:22 .
 - drwxr-xr-x 4 root root 4096 Jul 7 08:58 ..
 - rw-r--r-- 1 root root 11112 Oct 10 08:28 github.go
- `internal/github/` 目录包含一个文件：**github.go** (11,112 字节)

这是项目的核心文件之一，负责与 Github API 的交互。从我们之前的分析中，这个文件包含了：

主要功能模块

1. 数据结构定义：
 - `Issue` - Github Issue 结构体
 - `PullRequest` - PR 结构体
 - `Discussion` - Discussion 结构体
 - `Comment` - 评论结构体
 - `Reaction` - 反应结构体
 - `User` - 用户结构体
2. 核心函数：
 - `ParseURL()` - 解析 Github URL
 - `FetchIssue()` - 获取 Issue 数据
 - `FetchPullRequest()` - 获取 PR 数据
 - `FetchDiscussion()` - 获取 Discussion 数据
 - `FetchComments()` - 获取评论数据
 - `FetchReactions*()` - 获取反应数据
3. API 交互：
 - HTTP 客户端配置
 - 认证处理 (`GITHUB_TOKEN`)
 - 分页数据获取
 - 错误处理

这个文件是整个项目的获取数据层，负责从 Github API 拉取原始数据，然后传递给 `converter` 包进行 Markdown 格式转换。

> 我想将这个包里的所有调用Github RESTful API的地方换为基于Github GraphQL API的调用。

- 我来帮您将项目从GitHub RESTful API迁移到GraphQL API。这将是一个大的重构，让我们逐步进行。

- **Read**(internal/github/github.go)
└ Read 50 lines

- 根据分析，我发现了以下RESTful API调用点需要替换为GraphQL:

 现有RESTful API调用分析

主要API端点:

1. GET /repos/{owner}/{repo}/issues/{number} - 获取Issue
2. GET /repos/{owner}/{repo}/pulls/{number} - 获取PR
3. GET /repos/{owner}/{repo}/discussions/{number} - 获取Discussion
4. GET /repos/{owner}/{repo}/issues/{number}/comments - 获取评论
5. GET /repos/{owner}/{repo}/discussions/{number}/comments - 获取Discussion评论
6. GET /repos/{owner}/{repo}/issues/{number}/reactions - 获取Issue/PR反应
7. GET /repos/{owner}/{repo}/issues/comments/{id}/reactions - 获取评论反应

现在让我开始实现GraphQL版本:

- **Write**(internal/github/graphql.go)

Create file

internal/github/graphql.go

```
package github

import (
    "bytes"
    "encoding/json"
    "fmt"
    "io"
```

AI 会一次性读取 `api` 包下的所有 `.go` 文件，理解它们之间的相互关系，然后为你提供一个完整的、系统性的重构方案，并可以在你的授权后，直接生成修改后的所有文件内容。这是单个文件上下文完全无法做到的。

```
● Write(internal/github/graphql.go)
└─ User rejected write to internal/github/graphql.go
1 + package github
2 +
3 + import (
4 +     "bytes"
5 +     "encoding/json"
6 +     "fmt"
7 +     "io"
8 +     "net/http"
9 + )
10 +
11 + // GraphQLClient represents a GraphQL client for GitHub API
12 + type GraphQLClient struct {
13 +     token    string
14 +     baseURL  string
15 +     client   *http.Client
16 + }
17 +
18 + // NewGraphQLClient creates a new GraphQL client
19 + func NewGraphQLClient(token string) *GraphQLClient {
20 +     return &GraphQLClient{
21 +         token:    token,
22 +         baseURL:  "https://api.github.com/graphql",
23 +         client:   &http.Client{},
24 +     }
25 + }
26 +
27 + // GraphQLRequest represents a GraphQL request
28 + type GraphQLRequest struct {
29 +     Query      string `json:"query"`
30 +     Variables  map[string]interface{} `json:"variables"`
31 + }
32 +
33 + // GraphQLResponse represents a GraphQL response
34 + type GraphQLResponse struct {
35 +     Data interface{} `json:"data"`
```

@ 指令是 AI Agent 的**输入管道**。它以一种结构化的方式，将真实世界的、多维度的信息（本地文件、项目结构等）转化为 AI 可以理解的上下文。精通 @，是你从“把 AI 当玩具”到“把 AI 当工具”的第一步。

精通 ! 指令：赋予 AI 改造世界的力量

如果说 @ 是 AI 的“眼睛”，那么 ! 就是 AI 的“双手”。它打通了 AI 的“思考”与本地环境“执行”之间的壁垒，但同时又通过精巧的设计，保证了操作的绝对安全。

在对话中执行单行命令

这是 **!** 最直接的用法。你可以在与 AI 的对话流中，随时执行任何你想执行的 Shell 命令，而无需退出 Claude Code。

 复制代码

```
1 语法: ! <your_shell_command>
```

实战场景 1：即时验证与查询

你正在让 AI 帮你写一段操作 Redis 的 Go 代码，AI 建议使用 `go-redis` 库。你想确认一下本地是否已经安装了这个依赖。

 复制代码

```
1 > !go list -m github.com/redis/go-redis/v9
```

终端会直接执行这条命令，并返回结果。你可以立即知道依赖是否存在，然后继续你的对话。在我的 `issue2md` 下执行的结果如下：

```
! go list -m github.com/redis/go-redis/v9
└─ go: module github.com/redis/go-redis/v9: not a known dependency
```

注：输出上述结果是因为 `issue2md` 没有依赖 `go-redis`。

实战场景 2：将命令输出作为上下文

! 指令的输出，会自动成为下一轮对话的上下文，这是一个极其强大的特性。我们先让 Claude Code 做出一些改变，比如：

 复制代码

```
1 > @README-zh.md 基于README-zh.md，帮我生成一个繁体中文的README文件
```

```
2  | Read README-zh.md (54 lines)
3
4  • Write(README-zh-TW.md)
5  | Wrote 54 lines to README-zh-TW.md
6  # issue2md
7  一個命令列和網頁工具，用於將GitHub issue、discussion或pull request轉換為Markdown格式
8  >此倉庫中的大部分內容是由人工智慧生成的！
9  ## 命令列模式
10 ### 安裝issue2md命令列工具
11 ... +44 lines (ctrl+o to expand)
12
13 • 已基於 README-zh.md 成功生成繁体中文版本的 README 文件：README-zh-TW.md
14
15 主要轉換包括：
16 - 命令行 → 命令列
17 - 文件 → 檔案
18 - 安装 → 安裝
19 - 运行 → 運行
20 - 浏览器 → 瀏覽器
21 - 打开 → 開啟
22 - 按钮 → 按鈕
23 - 链接 → 連結
24 - 讨论 → 討論
25
26 檔案已保存在當前目錄下，可以直接使用。
```

假设你想让 AI 帮你写一个 Git Commit Message。

 复制代码

```
1 ! git add README-zh-TW.md
2 | (No content)
3
4 ! git diff --staged
5 | diff --git a/README-zh-TW.md b/README-zh-TW.md
6   new file mode 100644
7   index 00000000..33e9064
8   ... +58 lines (ctrl+o to expand)
```

终端会显示所有已暂存的代码变更。

 复制代码

```
1 > 根据上面的diff内容，帮我写一条符合Conventional Commits规范的commit message。
2
3
4 • 基于这个 diff 内容，我建议使用以下符合 Conventional Commits 规范的 commit message:
5
6 docs: add Traditional Chinese README (README-zh-TW.md)
7
8 这条 commit message 的说明:
9 - docs: 类型表示这是对文档的修改
10 - add 表示新增了一个文件
11 - Traditional Chinese README 简洁描述了文件内容
12 - (README-zh-TW.md) 指明了具体的文件名
13
14 如果你想更详细一些，也可以使用:
15
16 docs: add Traditional Chinese translation of README
17
18 - Convert simplified Chinese to traditional Chinese
19 - Add README-zh-TW.md for traditional Chinese users
20 - Maintain consistent formatting and structure
```

我们看到：Claude Code 会“阅读”上一条 **!** 命令的输出，并为你生成一条高质量的、与你的代码变更完全匹配的提交信息。

让 AI 提议并执行命令

这才是 **!** 指令的“AI 原生”用法。我们不再是自己执行命令，而是**让 AI 来决定何时需要执行命令，以及执行什么命令**。当 AI 认为需要通过执行 Shell 命令来获取信息或完成任务时，它会使用一个特殊的“工具调用”（Tool Call）来向你提议。

实战场景 3：AI 自主进行环境检查

你向 Claude Code 提了一个高层级的任务：“帮我分析这个项目的**外部模块依赖**”。AI 可能会这样回应（在 Claude Code 的界面中，这会以一个需要你确认的交互式卡片出现）：

> 帮我分析这个项目的模块依赖

● **Bash**(go mod tidy)

└ Running...

● **Read**(go.sum)

Bash command

go mod tidy

Generate go.sum file and download dependencies

Do you want to proceed?

> 1. Yes

2. Yes, and don't ask again for go mod tidy commands in /root/go/src/github.com/bigwhite/issue2md

3. No, and tell Claude what to do differently (esc)

我们看到 AI 给出了后续的本地操作建议，但此时需要你的授权。**这种“提议 – 审批”机制，是保障 **！** 指令安全的核心。**AI 永远不会在你不知情的情况下，擅自执行任何 `rm -rf /` 这样的危险命令。你，永远是那个掌握最终执行权的“司令官”。除非你在启动 Claude 时，开启了 SOLO 机制，即使用 `claude --dangerously-skip-permissions`，这样在新开启的 Claude Code 会话中将默认忽略掉任何需要你授权的环节。

一个重要的区别：没有沉浸式 Shell 模式

一些同学可能熟悉其他 AI Agent 工具提供的“沉浸式 Shell 模式”（即输入 **！** 后进入一个可以连续输入命令的子 Shell）。这里需要特别指出的是，**Claude Code 并未提供此功能**。如果你需要连续执行多条命令，最佳实践就是**逐行使用 **！** 前缀来执行它们**。这样做的好处是，AI 能够清晰地看到每一个独立的命令及其对应的输出，从而可以更精准地理解你的完整操作序列，并对每一步的结果做出响应。

组合的力量：完成你的第一个“感知 – 行动” workflow 闭环

现在，我们已经分别掌握了 AI 的“眼睛”(**@**)和“双手”(**！**)。是时候将它们组合起来，完成一个真正意义上的简化版的 AI 原生开发 workflow 闭环了。

任务：为一个 Go 项目添加新依赖，并编写一个使用该依赖的示例函数。

第一步：感知项目现状 (@**)**

```
> @go.mod
└─ Read go.mod (4 lines)

• Looking at your go.mod file, I can see this is the GitHub issue2md project:

- Module: github.com/bigwhite/issue2md
- Go version: 1.21.0
- Location: /root/go/src/github.com/bigwhite/issue2md

This appears to be a Go project for converting GitHub issues to markdown format. Would you like me to explore the codebase structure or help with something specific?
```

Claude Code 读取 `go.mod`，了解项目当前的依赖情况。

第二步：输入意图，让 AI 提议行动

```
> 我想为这个项目添加一个生成UUID的功能，请推荐一个流行的Go语言库。
```

- 对于Go语言的UUID生成，我推荐以下几个流行的库：
 1. **google/uuid** - 最流行的选择
 - Google维护，质量高
 - 支持UUID v1, v4, v5等多种版本
 - 纯Go实现，性能好
 - 使用简单：`uuid.New()`
 2. **segmentio/ksuid** - K-Sortable Unique ID
 - 可排序的UUID变体
 - 时间有序，适合某些场景
 - 更短的字符串表示
 3. **oklog/ulid** - Universally Unique Lexicographically Sortable ID
 - 字典序可排序
 - 时间戳+随机数组合
 - 适合分布式系统

对于你的issue2md项目，我建议使用 **google/uuid**，因为它是最标准、最广泛使用的UUID库。

要添加这个依赖，运行：

```
go get github.com/google/uuid
```

然后在代码中导入使用：

```
import "github.com/google/uuid"

// 生成UUID v4
id := uuid.New()
fmt.Println(id.String())
```

你希望我帮你集成这个UUID功能到项目中吗？

> 请在 `internal/utils/` 目录下创建一个 `uuid.go` 文件，里面包含一个名为 `NewUUID` 的函数，它返回一个新的 UUID 字符串。该函数基于 google/uuid 实现。

```
● Bash(go get github.com/google/uuid)
  └─ Running...
```

Bash command

```
go get github.com/google/uuid
Add google/uuid dependency to project
```

Do you want to proceed?

- > 1. Yes
- 2. Yes, and don't ask again for go get commands in /root/go/src/github.com/bigwhite/issue2md
- 3. No, and tell Claude what to do differently (esc)

你批准执行。

AI 知道要创建一个新文件，它会调用 `Write` 工具，并生成如下内容：

```
● Write(internal/utils/uuid.go)
```

Create file

```
internal/utils/uuid.go

package utils

import "github.com/google/uuid"

// NewUUID generates a new UUID v4 and returns it as a string
func NewUUID() string {
    return uuid.New().String()
}
```

Do you want to create uuid.go?

- > 1. Yes
- 2. Yes, allow all edits during this session (shift+tab)
- 3. No, and tell Claude what to do differently (esc)

你批准后，AI 完成代码写入，并提示你使用 go get 更新依赖：

● 已完成！我在 `internal/utils/uuid.go` 文件中创建了 `NewUUID` 函数，该函数：

- 导入了 `github.com/google/uuid` 库
- 实现了 `NewUUID()` 函数，生成 UUID v4 并返回字符串格式
- 代码简洁易用

使用方法：

```
import "github.com/bigwhite/issue2md/internal/utils"
```

```
id := utils.NewUUID() // 返回类似 "550e8400-e29b-41d4-a716-446655440000" 的字符串
```

需要先安装依赖：`go get github.com/google/uuid`

> █

? for shortcuts

第三步：更新依赖 (!)

我们使用 `!` 指令，参考 Claude Code 给出的建议执行 `go get`，以更新 `go.mod` 中的依赖：

```
! go get github.com/google/uuid
└─ go: added github.com/google/uuid v1.6.0

> 
```

第四步：完成并记录 (!)

假设测试通过了。我们希望把所有这些变更 (`go.mod`, `go.sum`, 新增的 `uuid.go`) 暂存，生成一条 `commit message`，并提交：

```
> 测试通过了。请帮我把所有这些变更 (go.mod, go.sum, 新增的 uuid.go) 暂存，并生成一条 commit message。
```

```
● Bash(git status)
└─ On branch main
   Your branch is up to date with 'origin/main'.
   ... +14 lines (ctrl+o to expand)

● Bash(git diff)
└─ diff --git a/go.mod b/go.mod
   index c222d95..dd118d2 100644
   --- a/go.mod
   ... +7 lines (ctrl+o to expand)

● Bash(git log --oneline -5)
└─ 235d768 Merge pull request #13 from drupol/push-yplkqllvusot
   91afc42 issue2md: transform the output header into a list
   81d38b6 Merge pull request #11 from drupol/push-yqrrzzwovsqy
   ... +2 lines (ctrl+o to expand)

● Bash(git add go.mod go.sum internal/uuid.go)
└─ Running...
```

Bash command

```
git add go.mod go.sum internal/uuid.go
Stage go.mod, go.sum, and internal/uuid.go directory
```

Do you want to proceed?

- ```
> 1. Yes
2. Yes, and don't ask again for git add commands in /root/go/src/github.com/bigwhite/issue2md
3. No, and tell Claude what to do differently (esc)
```



#### Bash command

```
git commit -m "feat: add UUID generation utility"

- Add google/uuid v1.6.0 dependency
- Create internal/utils/uuid.go with NewUUID function
- NewUUID generates UUID v4 strings for use throughout the project

🤖 Generated with [Claude Code](https://claude.com/claude-code)

Co-Authored-By: Claude <noreply@anthropic.com>"
Create commit with UUID feature addition

Do you want to proceed?
> 1. Yes
2. Yes, and don't ask again for git commit commands in /root/go/src/github.com/bigwhite/issue2md
3. No, and tell Claude what to do differently (esc)
```

- 完成！已成功暂存所有变更并创建了提交：

#### 提交信息：

feat: add UUID generation utility

- Add google/uuid v1.6.0 dependency
- Create internal/utils/uuid.go with NewUUID function
- NewUUID generates UUID v4 strings for use throughout the project

#### 包含的文件：

- go.mod - 添加了 google/uuid 依赖
- go.sum - 新生成的依赖校验和文件
- internal/utils/uuid.go - 新创建的UUID工具文件

提交已成功创建，当前分支领先 origin/main 1 个提交。

看到了吗？从分析需求，到修改环境，再到编写代码，最后版本控制，整个流程一气呵成。你全程扮演的是一个**提出意图、审查决策**的“架构师”角色，而所有具体的、繁琐的“感知 - 行动”循环，都由 AI 高效地完成了。

## 本讲小结

今天，我们深入学习了所有 Coding Agent 的“通用语言”——以 @ 为代表的**上下文注入**和以 ! 为代表的**Shell 执行**。这不仅仅是两个命令，它们共同构成了一套全新的人机协作哲学。



首先，我们建立了 AI“眼 + 手”的心智模型：@ 负责感知世界，! 负责改造世界。接着，我们系统性地学习了 @ 指令的两种主要用法：引用单个文件和引用整个目录（及其智能的 Git 感知能力），它解决了 AI 的“失忆症”。然后，我们深入了 ! 指令的两种核心用法：手动执行以快速验证，以及让 AI 提议并执行以驱动工作流。

最后，通过一个完整的实战案例，我们将 @ 和 ! 组合起来，体验了一个流畅的“感知 - 行动”工作流，初步感受了作为“AI 工作流指挥家”是一种怎样的体验。

掌握了 @ 和 !，你就拥有了与 AI 进行结构化、高效协作的基础能力。可以说，你已经正式踏入了 AI 原生开发的大门。然而，@ 指令只能提供“一次性”的上下文。如果有些知识，比如项目的编码规范、架构原则，你想让 AI 在整个会话中、甚至在所有未来的会话中都“永远记住”，该怎么办呢？

这就是我们下一讲要探讨的主题：上下文的艺术。我们将深入 CLAUDE.md 和 agents.md，为你揭示如何为 AI 打造一个持久的、分层的“长期记忆系统”。

## 思考题

今天我们学习了 ! 指令，可以将命令的输出作为下一轮对话的上下文。请你设想一个场景：你如何利用这个特性，仅通过 ! 指令和自然语言，让 AI 帮你找出项目中所有超过 500 行的



Go 文件，并对其中最大的那个文件进行代码复杂度分析？请描述你的操作步骤。

欢迎在评论区分享你的“指令链”，看看谁的方案最高效、最优雅！如果你觉得有所收获，也欢迎你分享给需要的朋友，我们下节课再见！

## AI智能总结

1. AI的核心交互模型建立在`@`和`!`两个符号之上，分别代表上下文和行动，成为与高级Coding Agent对话的“通用语言”。
2. `@`指令提供上下文，让AI能够主动观察整个项目的“全局架构师”，解决“上下文摩擦力”问题。
3. `!`指令执行动作，将AI的思考直接转化为真实世界的影响，无缝地融入到对话流之中。
4. AI的交互模型使得交互更像是双向的、协作式的对话，类似于与人类同事进行结对编程的协作模式。
5. AI的路径自动补全功能和对文件内容的理解使得与AI的交互更加高效。
6. AI能够根据代码内容生成高质量的单元测试，并提供后续的建议操作。
7. 多阶段构建的优势和最佳实践展示了AI对现代Go应用容器化的理解和实践能力。
8. AI的交互模型为开发者提供了更高效、更智能的协作方式，使得开发过程更加流畅和高效。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

## 全部留言 (19)

最新 精选



6点无痛早起学习的和...

2025-12-03 来自北京

有个问题，在 cc 中，当我提到某个类的时候，他习惯于用 grep 去搜索，那这里直接用@的区别是什么？

在实操过程中，发现！其实很少用，都是 cc 自动去执行课后作业：

方案一：直接给 cc，@xx(是个目录)帮我找出这个目录下所有超过 500 行的 Go 文件（包括子目录等），并对其中最大的那个文件进行代码复杂度分析？

方案二：先 !tree，然后直接发给 cc，帮我找出这个上面这些所有超过 500 行的 Go 文件，并对其中最大的那个文件进行代码复杂度分析？

感觉这些内容还是比较偏基础，期待老师后面的内容

作者回复：区别在于“准确性”与“token 消耗”。

自动 grep 是 AI 的一种“探索”行为。当它不知道确切位置时，会去搜。但这会消耗额外的步骤和 token，而且搜出来的可能只是片段，或者搜到一堆不相关的文件，容易造成“上下文污染”。

手动 @ 是你作为“指挥家”的精准指引。你直接把确切的、完整的文件喂给它，AI 就不需要瞎猜乱找了，理解更准，效率更高。当你明确知道要改哪个文件时，一定要用 @。

此外，你提到的! 很少用，其实是 AI 越来越智能的表现。当你在 Prompt 里说“运行测试”，AI 确实会自动调用 Bash(go test)，这在本质上和 ! 是一样的。

而! 的价值在于“主动控制”。比如：你想在对话中间插一句“我先看看当前目录有啥”，用 !ls 最快，不用废话让 AI 去“决定”调用工具。



👍 12



**geek**

2025-12-17 来自浙江

请问老师，@代码文件时，会输出一些对文件的理解，这个输出是 Claude code cli 输出的还是智谱大模型输出的？感觉应该是大模型输出的，不太确定

作者回复: claude code cli会读取文件，并将文件内容发给大模型理解，claude code cli会输出大模型理解后的响应内容。

共 2 条评论 >

👍 1



**望星空**

2025-12-12 来自广东

老师，CC 默认是取多少条历史对话?可不可以配置的？

作者回复: 据我了解，Claude Code 并没有一个固定的“条数”限制，但它受限于Context Window（上下文窗口）的大小，不同大模型的窗口大小不同。

Claude Code 会尽可能多地保留历史对话，直到接近 Token 上限。

当上下文过长时，它会触发 Auto-summarization（自动摘要），将早期的对话压缩成摘要保留，释放空间给新对话。

你可以通过 `/compact` 命令手动触发这个压缩过程，或者使用 `/clear` 彻底清空历史（保留文件上下文），来主动管理你的上下文预算。



1



**bearlu**

2025-12-01 来自广东

@，能不能指向一个API接口网站？

作者回复: 应该不行，只是能接本地路径。

共 2 条评论 >



1



**南**

2025-11-30 来自浙江

老师，一个完整的前后端项目，如果出错了，比如登陆失败，如何让ai来正确排查呢

作者回复: 可以啊。在后续会有类似内容的针对性讲解。



1



**rOMeO罗密欧**

2025-11-30 来自广东

这些代码例子有没有github repo可以给我们下载试试？

作者回复: 简单的代码，文章里都有。

基于已有项目的，你可以git clone我的github.com/bigwhite/issue2md做试验。

后续完整实战内容时，会有一个项目示例提供给大家[手动抱拳]。



1



**AKA三皮**

2025-11-28 来自江苏

求问各位大佬，在codex中，@ 是否支持“文件夹”级别的查找～ 我试了好像不行

作者回复: 哪位熟悉codex的大神给解答一下😄

共 2 条评论 >

👍 1



**zhyustar**

2026-01-12 来自上海

老师你好，请教下如果是一个大工程是否也可以用@来导入整个工程

作者回复: 理论上可以，但工程上极其不推荐。

@整个工程 会把工程所有文件的全部代码都喂给 AI。即使是 200k 的上下文窗口也可能瞬间爆满。这不仅极慢，而且极贵。

此外，上下文太长，AI 会“迷失”，会注意力分散。它不知道你到底关注哪个模块，幻觉概率大增。

因此，实践中，我们还是只 @ 你当前任务相关的目录（如 @internal/user/）。

或者将项目的布局结构信息提供给AI，比如用 !tree -L 2 或 ls 让 AI 先看目录结构，然后让 AI 自己决定需要读取哪个具体文件的详情。即所谓的“先看地图，再看房间”。这也是一种渐进式“披露”的策略。



**victoriest**

2026-01-09 来自湖北

老师你好，当我使用 官方提供的VS code插件时，用！执行命令似乎没有生效；请问是需要有什么额外的设置还是插件限制呢

作者回复: 这个我还没用过。

不过我猜是不是VS Code 插件运行在受限的沙箱环境中，为了安全，默认不提供直接的 Shell 执行权限所致？



**batman**

2025-12-31 来自福建

如果是历史项目，需要再多个不同的文件间跳转切换，感觉用命令行非常不方便

作者回复: 如果你是指你自己查看文件不方便，那完全没必要把自己锁死在终端里。可以双屏协作：在 IDE（VS Code）里看代码、跳文件（利用 IDE 强大的跳转功能），在终端里指挥 AI 干活。

